```
!pip install opencv-python-headless
```

```
Requirement already satisfied: opencv-python-headless in /usr/local/lib/python3.12/dist-packages (4.13.0.92)
Requirement already satisfied: numpy>=2 in /usr/local/lib/python3.12/dist-packages (from opencv-python-headless) (2.0.2)
```

```
import cv2
import numpy as np
import glob
import os
from google.colab import files
from google.colab.patches import cv2_imshow
```

```
uploaded = files.upload()
```

Choose files 5 files

CAM5.jpg(image/jpeg) - 123355 bytes, last modified: 15/03/2026 - 100% done
CAM4.jpg(image/jpeg) - 87879 bytes, last modified: 15/03/2026 - 100% done
CAM3.jpg(image/jpeg) - 108604 bytes, last modified: 15/03/2026 - 100% done
CAM2.jpg(image/jpeg) - 87197 bytes, last modified: 15/03/2026 - 100% done
CAM1.jpg(image/jpeg) - 146667 bytes, last modified: 15/03/2026 - 100% done

```
Saving CAM5.jpg to CAM5 (1).jpg
Saving CAM4.jpg to CAM4 (1).jpg
Saving CAM3.jpg to CAM3 (1).jpg
Saving CAM2.jpg to CAM2 (1).jpg
Saving CAM1.jpg to CAM1 (1).jpg
```

```
os.makedirs("Checkerboard", exist_ok=True)

for file in uploaded.keys():
    os.rename(file, "Checkerboard/" + file)

print("Images stored in /content/Checkerboard")
```

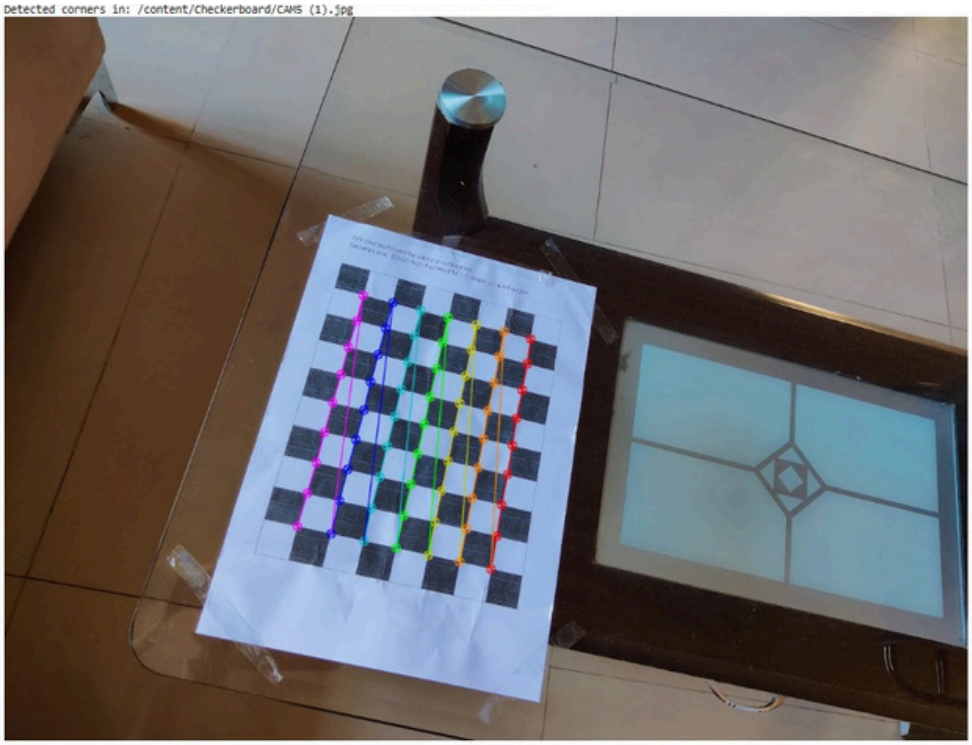
```
Images stored in /content/Checkerboard
```

```
# Chessboard configuration
num_squares_x = 8
num_squares_y = 10

num_interior_corners_x = num_squares_x - 1
num_interior_corners_y = num_squares_y - 1

checker_width = 0.020

path = "/content/Checkerboard"

def calibrate():

    termination_criteria = (
        cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER,
        30,
        0.001
    )

    object_points_3D = np.zeros(
        (num_interior_corners_x * num_interior_corners_y, 3),
        np.float32
    )

    object_points_3D[:, :2] = np.mgrid[
        0:num_interior_corners_y,
        0:num_interior_corners_x
    ].T.reshape(-1, 2)

    object_points_3D = object_points_3D * checker_width

    object_points = []
    image_points = []

    images = glob.glob(os.path.join(path, "*.jpg"))

    for image_file in images:

        image = cv2.imread(image_file)
        gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

        success, corners = cv2.findChessboardCorners(
            gray,
            (num_interior_corners_y, num_interior_corners_x),
```

```
            None
        )

        if success:

            object_points.append(object_points_3D)

            corners_2 = cv2.cornerSubPix(
                gray,       corners,
                (11,11),    (-1,-1),
                termination_criteria


            )

            image_points.append(corners_2)

            cv2.drawChessboardCorners(
                image,
                (num_interior_corners_y, num_interior_corners_x),
                corners_2,
                success
            )

            print("Detected corners in:", image_file)
            cv2_imshow(image)

    ret,mtx,dist, rvecs, tvecs = cv2.calibrateCamera(
        object_points,
        image_points,
        gray.shape[::-1],
        None,
        None
    )

    cv_file = cv2.FileStorage(
        "calibration_chessboard_web.yaml",
        cv2.FILE_STORAGE_WRITE
    )

    cv_file.write("K", mtx)
    cv_file.write("D", dist)
    cv_file.release()

    print("\nCamera Matrix:\n", mtx)
    print("\nDistortion Coefficients:\n", dist)
```

```
calibrate()
```

Detected corners in: /content/Checkerboard/CAM5 (1).jpg



Detected corners in: /content/Checkerboard/CAM4 (1).jpg

Detected corners in: /content/Checkerboard/CAM3 (1).jpg

```
Camera Matrix:
 [[9.61771876e+02 0.00000000e+00 6.26958263e+02]
 [0.00000000e+00 1.01147024e+03 4.35731693e+02]
 [0.00000000e+00 0.00000000e+00 1.00000000e+00]]

Distortion Coefficients:
 [[ 0.20134596 -2.10479502 -0.00460595  0.00549483  4.33434236]]
```